

CI Integration with Artifact Repositories

Wiring Maven / Gradle builds and Nexus into a real, automated CI pipeline

IN THIS DOCUMENT

- The end-to-end CI → artifact repo flow
- Maven configuration for publishing to Nexus
- Authentication & secrets handling
- Key CI best practices

Prepared for internal training · Topic 3 of 3

1 The End-to-End Flow

This is where build automation, dependency management, and Nexus all come together inside a CI pipeline (Jenkins, GitLab CI, GitHub Actions, etc.):

1. Checkout code
2. Resolve dependencies ← pulled from Nexus, not the open internet
3. Compile & test
4. Package artifact (JAR / WAR / container image)
5. Publish artifact to Nexus → mvn deploy / gradle publish
6. Downstream consumption ← deployment stage, or another team's build, pulls that exact versioned artifact back out of Nexus

2 Maven Configuration for CI

Authentication (settings.xml)

Credentials are supplied via CI environment variables/secrets — never hardcoded into the repository.

```
<server>
  <id>nexus-releases</id>
  <username>${env.NEXUS_USER}</username>
  <password>${env.NEXUS_PASS}</password>
</server>
```

Distribution Management (pom.xml)

This tells Maven where to publish release and snapshot builds:

```
<distributionManagement>
  <repository>
    <id>nexus-releases</id>
    <url>https://nexus.company.com/repository/maven-releases/</url>
  </repository>
  <snapshotRepository>
    <id>nexus-snapshots</id>
    <url>https://nexus.company.com/repository/maven-snapshots/</url>
  </snapshotRepository>
</distributionManagement>
```

3 Key CI Best Practices

Practice	Why It Matters
Never hardcode credentials	Use the CI platform's secret manager (Jenkins Credentials Store, GitHub Secrets, etc.)
Version artifacts consistently	Semantic versioning for releases, build-number-based for snapshots
Fail fast on unresolved dependencies	Don't silently fall back to public repos in production pipelines — it breaks reproducibility and bypasses security control
Cache at the CI-runner level too	Speeds up builds further, layered on top of Nexus's own caching

Big picture: Maven/Gradle handle the "how do I build this," Nexus handles "where does it live," and CI is the glue that makes the whole cycle — resolve → build → publish → consume — fully automated and repeatable.

Part 3 of 3 — CI Integration with Artifact Repositories. See Part 1 for build automation basics and Part 2 for Nexus repository setup.